Assignment no. 3

Natural Language Processing
NLP4IL (AIS.ba VZ)
WS 2026

For this assignment sheet we will be using the 20 newsgroups text dataset, which can be easily loaded with sklearn:

```
from sklearn.datasets import fetch_20newsgroups

cats = ['sci.space', 'comp.graphics', 'rec.sport.hockey', 'alt.atheism']
data = fetch_20newsgroups(subset='train', categories=cats, remove=('headers','footers','
    quotes'))
X_text, y, target_names = data.data, data.target, data.target_names
```

Exercise 1 Bag of Words & TF-IDF

In the first exercise, compare the Bag of Words and TF-IDF representations for the 20 newsgroups text dataset by taking a look at the similarities between the 5 news categories.

Tasks

- Fit CountVectorizer (Bag of Words) and TfidfVectorizer (TF-IDF). Make sure to remove the stopwords.
- Top terms per class based on TF-IDF: For each category, compute the top-10 terms by average TF-IDF over the whole corpus.
- Similarity: Pick a random message for each category. For those messages, compute the cosine similarity of the TF-IDF and BoW repesentations. Compare which representation separates categories better.
- Discuss in exercise: When does TF-IDF outperform BoW?

Exercise 2 N-Grams and Feature Explosion

Generate example n-grams and briefly examine why their number grows rapidly and how this affects real-world NLP.

Tasks

- Generate all bigrams and trigrams of the sentence ”The quick brown fox jumps over the lazy dog”.
- Discuss the concept of feature explosion:
  - How many possible bigrams exist in a vocabulary of size V?
  - Why does this become a problem in real-world NLP?
  - Discuss in exercise: Considering the problems, what are the advantages?

Exercise 3 Word2Vec Concepts: CBOW and Skip-Gram

Revisit how Word2Vec learns word relationships by predicting contexts. Demonstrate the two variants by forming training pairs from a simple sentence.

Tasks

- Explain in your own words the difference between CBOW and Skip-Gram.
- Given the sentence: "The quick brown fox jumps over the lazy dog." and using a context window size of 2, list the (input $\rightarrow$ target) training pairs generated by CBOW and Skip-Gram.

Exercise 4 CBOW vs Skip-Gram (Train & Probe)

You will train both Word2Vec architectures on the same corpus and compare the semantic neighborhoods they produce. This highlights how each model behaves on small datasets and which one captures meaning more reliably.

Tasks

- Preprocess: tokenize with gensim.simple_preprocess, remove stopwords.
- Train CBOW (sg=0) and Skip-Gram (sg=1) with modest settings (e.g., vector_size=100, window=5, min_count=5, epochs=10)
- For 3 query words (e.g., space, graphics, hockey), list top-10 most similar words for each model. Compare qualitatively (topics? noise?).
- Discuss in exercise: Which works better on small data? Why?

Exercise 5 FastText & Embedding Visualization

FastText enriches word embeddings with subword information, improving representations for rare words. You will train a model and visualize its embeddings to observe clusters and outliers in 2D space.

Tasks

- Train FastText on the same sentences with subword information (min_n=3, max_n=6).
- Build a matrix of word vectors and reduce to 2D via PCA and t-SNE.
- Plot points and label them based on category. Interpret clusters and outliers.
- Pick 15–30 representative words across categories to have a more detailed comparison. Process and plot them as well.

When using Large Language Models (eg. ChatGPT, Copilot, ...) to produce output that is handed in, also state the prompt and model used in the submission. Submissions that are not marked and are noticeably AI generated will be graded with zero points!

Submission: electronically via Moodle; Deadline: Mo, Mar 23, 2026, 23:59.